BONAFIDE CERTIFICATE

Certified that this mini project report titled "CloudLedger - Cloud Resource Cost Analyzer" is the bonafide work of "Abhinav Deshwal, UID: 26MCC20019". The work was carried out under my supervision during the MCA course. The project source includes a Python Streamlit application and Oracle SQL and PL/SQL scripts for cloud resource cost tracking.

The work presented in this report is based on the supplied project files. Student and supervisor details may be filled before submission.

Supervisor	**Head of Department**
Signature: _____ Name: Ms. Tanisha Designation: Assistant Professor	Signature: _____ Name: Dr. Amanpreet Kaur Department: MCA

Date of project examination: 22nd Oct, 2026

Internal Examiner	**External Examiner**
Name: Ms. Tanisha	Name: _____
Ecode: E20679	Ecode: _____

TABLE OF CONTENTS

Bonafide Certificate 2
Table of Contents and List of Figures 3
Acknowledgement and Abstract 4
Chapter 1: Introduction 5
Chapter 2: Background Study 6
Chapter 3: Design Flow and Process 7
Chapter 4: Implementation 8
Chapter 5: Database and Validation 11
Chapter 6: Conclusion and References 14
GitHub Repository ……..... 15

LIST OF FIGURES

Figure 1: System architecture 7
Figure 2: CloudLedger Dashboard page 8
Figure 3: Resource Management page 9
Figure 4: Clock and budget settings 10
Figure 5: Budget controls and alert evaluation 10
Figure 6: GitHub Repository of CloudLedger 15

ACKNOWLEDGEMENT

I express my sincere gratitude to Ms. Taisha for valuable guidance, clear suggestions, and encouragement during this mini project. The support received helped me understand the work step by step and present it in a proper report format.

I am thankful to the Head of Department and all faculty members of Master f Computer Applications for giving me the opportunity and facilities to complete this work. Their teaching helped me connect Python, a web interface, and an Oracle database in one small project.

I also thank my family and friends for their patience and support. This project has improved my confidence in planning a simple database-based web application and explaining its result in clear language.

ABHINAV DESHWAL

ABSTRACT

CloudLedger is a web application for keeping track of cloud-resource cost. It uses a Python Streamlit interface with an Oracle database. The database stores resource details, usage sessions, budget settings, billing cycles, and alerts. PL/SQL packages and functions are used for the main database work.

The application has three pages: Dashboard, Resources, and Settings. The supplied live screens show that the application was connected to Oracle. The Dashboard gives a quick view of current-cycle cost, budget use, running resources, time information, and stored resource records. The Resources page shows resource details and options for creating, editing, stopping, or deleting a selected resource. The Settings page provides demo-clock and budget controls.

This report explains the actual project in simple language and uses genuine application screenshots in the implementation chapter. The values shown in figures are the values visible at the time of capture. A login page is not part of the present project; basic login guidance is included only as a future-work reference.

CHAPTER 1: INTRODUCTION

1.1 Project Scope

Cloud services can have many resources such as virtual machines, storage, and compute services. Each running resource can create a cost. CloudLedger is made to store this information in one place. It gives a small dashboard for checking the current cost and budget condition.

1.2 Need for the Project

A user may forget which resources are running or how much they cost. This can make it hard to control spending. The project keeps a name, type, rate, status, and description for each resource. It also saves usage sessions so cost can be calculated from time and rate.

1.3 Objectives

No.	Objective
1	Store cloud resource details in Oracle.
2	Start and stop resources through a simple dashboard.
3	Calculate current-cycle cost from usage sessions.
4	Save a budget and create warning or exceeded alerts.
5	Provide a safe SQL verification script.

1.4 Technologies Used

Technology	Use in the project
Python	Application logic and database connection layer.
Streamlit	Web dashboard and forms.
Oracle Database	Data storage and SQL/PLSQL processing.
PL/SQL	Packages, function, and triggers.
python-oracledb	Oracle connection from Python.

CHAPTER 2: BACKGROUND STUDY

2.1 Existing Method

A simple way to track cost is to write resource details in a sheet. This may not show the current status or calculate time-based cost correctly. CloudLedger uses a database so the information can be saved in related tables.

2.2 Main Ideas Used

The project uses a resource table for cloud resource details. When a resource starts, a usage session is created. When it stops, the end time is saved. The billing package checks the time overlap for the current budget cycle and adds the cost.

2.3 Problem Definition

The problem is to make a small system that can save cloud resource information and show cost and budget details. The system must avoid hardcoded dashboard values. It must read its values from Oracle records.

2.4 Features Found in the Source

Area	Implemented feature
Resources	Create, edit, delete, start, and stop actions.
Billing	Current-cycle cost from session time and hourly rate.
Clock	Speed, pause, resume, and reset controls.
Budget	Amount, cycle type, warning percentage, and alert check.
Safety	Input checks, constraints, and a non-destructive verification script.

2.5 Limitation

The project needs a reachable Oracle service and a configured application user. The provided README states that the SQL scripts must be run in order before the dashboard can load records. No sample resource data is inserted by the supplied schema script.

CHAPTER 3: DESIGN FLOW AND PROCESS

3.1 System Design

The user works through the Streamlit dashboard. The dashboard calls helper functions in the Python files. The helper functions connect to Oracle and run SQL or stored procedures. Oracle stores the data and returns records to the dashboard.

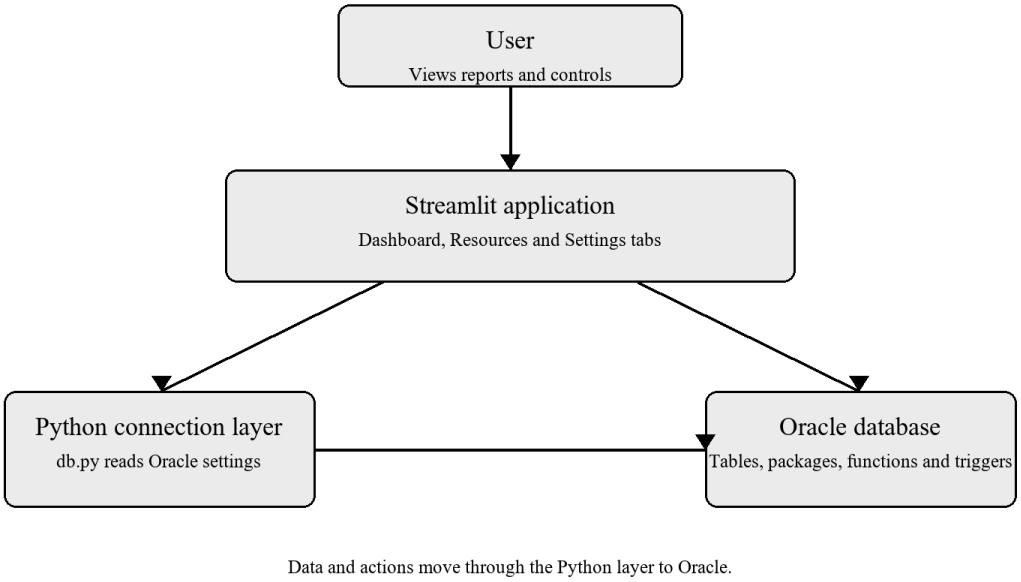


Figure 1. System architecture based on the supplied project source.

3.2 Data Flow

For a resource action, the user selects or enters values in the dashboard. The application calls a PL/SQL procedure. Oracle saves the change and the dashboard is refreshed. For cost, the application asks the billing function for the current value and displays the returned result.

CHAPTER 4: IMPLEMENTATION

4.1 Dashboard

Figure 2 shows the CloudLedger Dashboard after it connected to Oracle. The page gives a quick view of current-cycle cost, budget consumed, and the number of running resources. It also shows local time, simulated time, budget details, and a small resource summary.

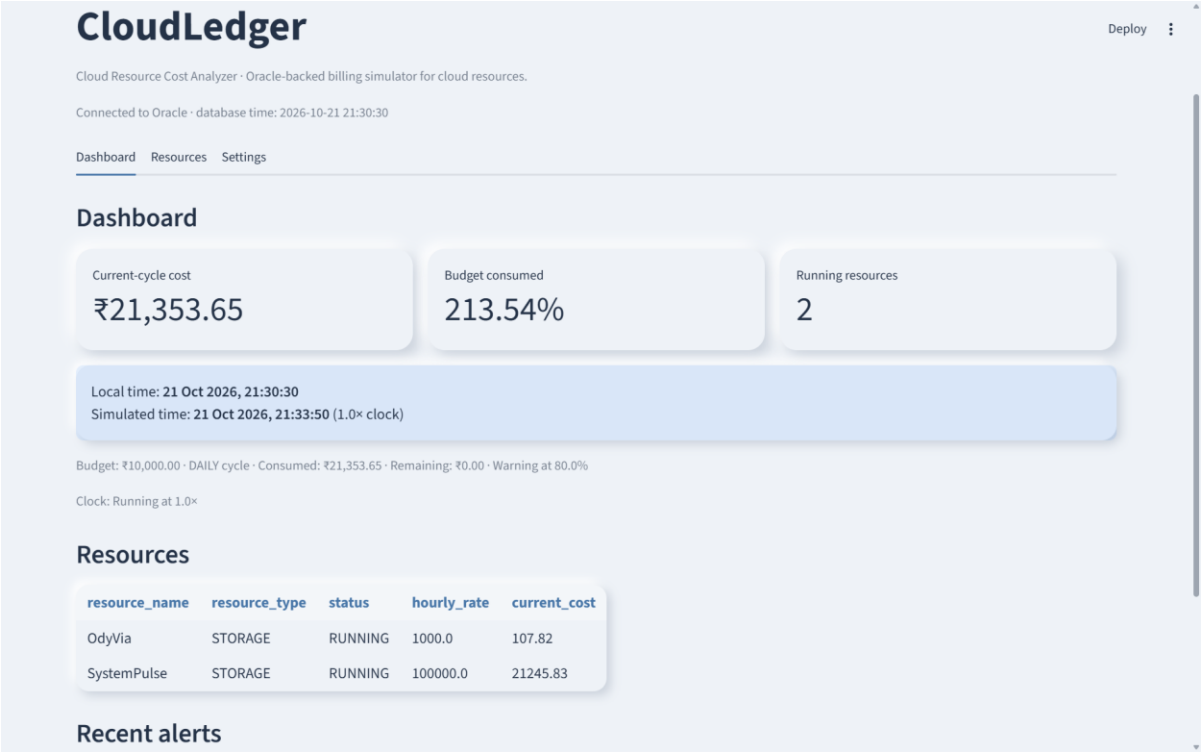


Figure 2. CloudLedger Dashboard page showing live cost and resource summary.

4.2 Dashboard Page Features

At the time of capture, the page showed two running storage resources. It also displayed a daily budget of ₹10,000.00, an 80% warning level, and budget use above the saved budget. This layout helps the user notice the cost condition without opening each resource record.

4.3 Resource Management

Figure 3 shows the Resource Management page. The table displays resource ID, name, type, hourly rate, status, description, and current cost. A user can open Create resource, select a saved record, and then use the available stop, delete, or edit controls.

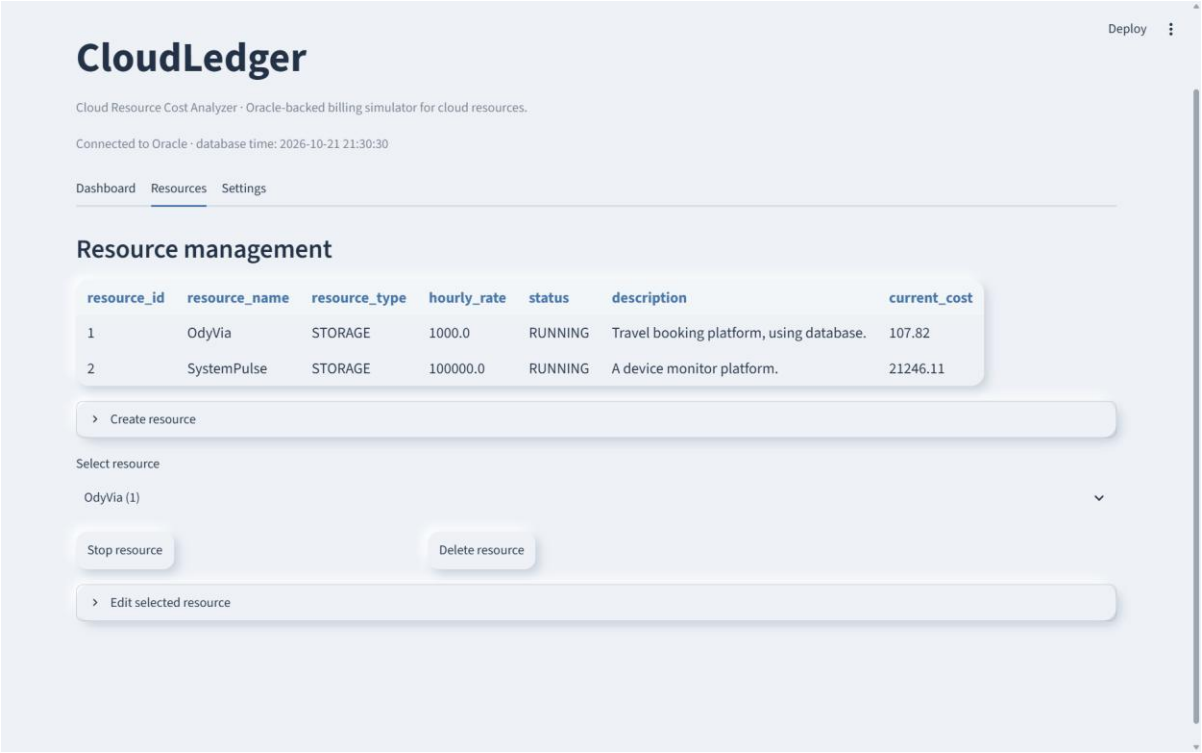


Figure 3. CloudLedger Resource Management page with saved resource records and controls.

4.4 Clock and Budget Settings

The Settings page controls simulated time and budget information. Figure 4 shows the accelerated demo clock with speed, apply, pause, and reset controls. It also shows budget amount, cycle, warning threshold, and currency fields.

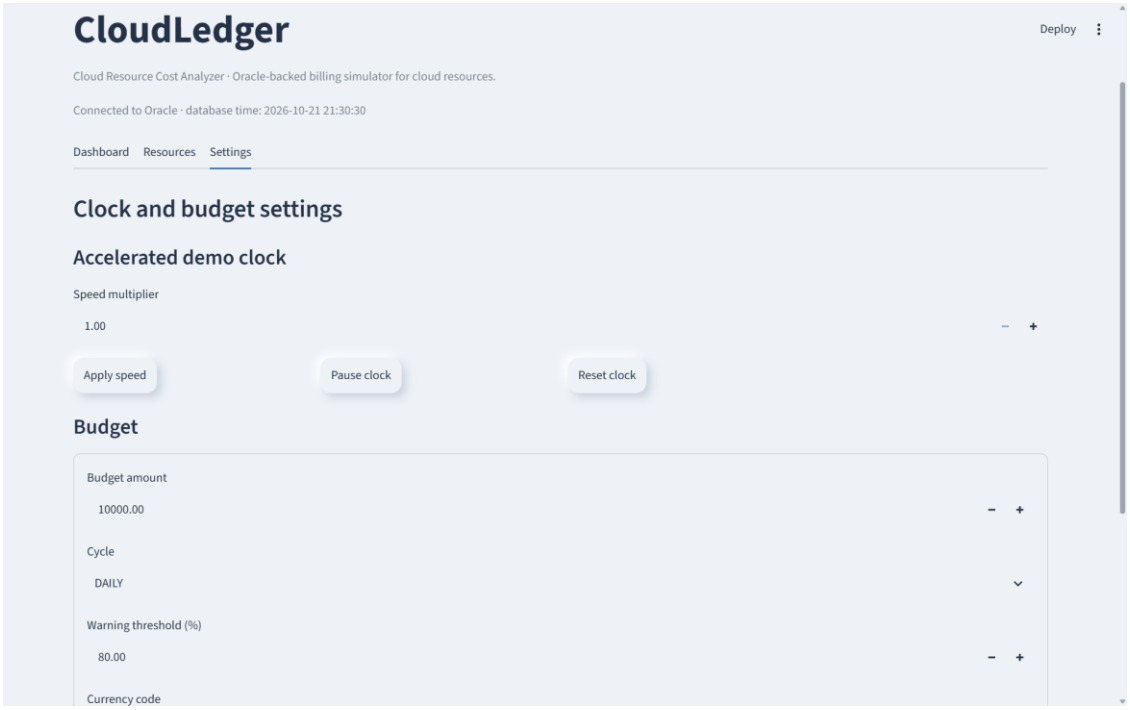


Figure 4. CloudLedger Clock and Budget Settings page.

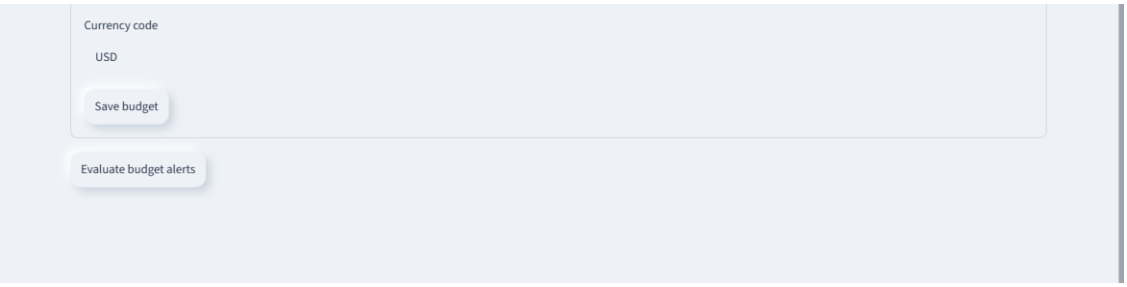


Figure 5. Budget controls for currency, saving settings, and evaluating alerts.

CHAPTER 5: DATABASE IMPLEMENTATION

5.1 Tables and Rules

The first SQL script creates sequences and tables. The important tables are CLOUD_RESOURCES, USAGE_SESSIONS, BUDGET_SETTINGS, DEMO_CLOCK, BILLING_CYCLES, and BUDGET_ALERTS. Primary keys, foreign keys, unique rules, and check constraints are used to protect data quality.

Table	Purpose
CLOUD_RESOURCES	Keeps the name, type, hourly rate, status, and description of each resource.
USAGE_SESSIONS	Stores the start and stop time used for cost calculation.
BUDGET_SETTINGS	Stores budget amount, cycle, warning level, and currency.
DEMO_CLOCK	Stores the simulated-time settings used in the demonstration.
BUDGET_ALERTS	Keeps warning or exceeded-budget messages.
BILLING_CYCLES	Stores the start and end dates of a billing cycle.

5.2 Data Integrity

The tables use IDs to identify records. A usage session is linked to a resource, so the database can relate the time record to the correct hourly rate. Rules such as valid status, unique names, and required values help prevent incorrect data from being saved.

5.3 Database Logic

The second SQL script contains packages for the demo clock, cloud resources, billing, and budgets. The billing package finds a cycle period, checks usage-session time, and returns a rounded cost value.

Part	Simple purpose
Demo clock	Changes, pauses, or resets the simulated demonstration time.
Resource management	Creates, edits, starts, stops, or deletes resource records.
Billing	Calculates the current-cycle cost from saved usage time and hourly rate.
Budget	Saves budget details and checks whether an alert is needed.

5.4 Basic Login Reference for Future Work

The present CloudLedger project does not include a login page. If login is added later, each user should have a unique user name and a password that is stored safely as a one-way password hash. The application should check the user before showing protected pages and should give only the required access. This is a future improvement, not a feature shown in the supplied screens.

5.5 Validation from Captured Screens

The following checks are based on the supplied live CloudLedger screenshots. They are used as visual evidence of the displayed pages and controls.

Check	Evidence visible in the screenshot	Result
Oracle connection	The page header says that the application is connected to Oracle and shows database time.	Observed
Dashboard metrics	Current-cycle cost, budget consumed, and running-resource count are displayed.	Observed
Resource records	Two resource rows show status, rate, description, and current cost.	Observed
Clock controls	Speed, apply, pause, and reset controls are visible.	Observed
Budget actions	Save budget and Evaluate budget alerts buttons are visible.	Observed

5.6 Verification Plan

The supplied file 05_verify.sql is a non-destructive verification script. It checks the connected schema, required tables, required PL/SQL objects, compilation errors, and basic package calls. It does not insert, update, or delete data.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Test Status

Check	Evidence	Status in this report
Python source	app.py and db.py were inspected.	Reviewed
Schema scripts	01_tables.sql through 05_verify.sql were inspected.	Reviewed
Live interface	The supplied Dashboard, Resources, and Settings screens show a connected Oracle session.	Observed
Database values	The figures display values at the time the screens were captured.	Observed

This report uses the supplied live screenshots for page-level evidence. The displayed costs and amounts should be treated as the captured session values, because they can change when the resource state, time, or budget changes.

6.2 Conclusion

CloudLedger is a small Oracle-backed cloud cost project. Its source has a clear separation between the Streamlit interface, Python connection code, SQL tables, PL/SQL packages, function, and triggers. The system is designed to calculate cost from saved usage sessions instead of fixed sample values.

The project includes the main actions needed for a basic cloud resource cost analyzer: resource management, a simulated clock, budget settings, billing logic, and alert evaluation. The verification script is useful because it checks objects without changing stored data.

6.3 Future Work

Future improvement	Simple purpose
Charts	Show cost change over time.
Basic login and roles	Add protected access in a future version; this is not in the current project.
Export report	Save selected cost data in a file.

More alerts	Add email or on-screen notifications after proper configuration.

REFERENCES

1. Supplied project source: CloudLedger README.md, app.py, and db.py.

2. Supplied project source: sql/01_tables.sql to sql/05_verify.sql.

3. Oracle Database documentation, SQL and PL/SQL reference material.

4. Streamlit documentation, Python web application framework reference material.

5. OWASP Foundation, Authentication Cheat Sheet, basic login and password-security reference material.

GITHUB REPOSITORY

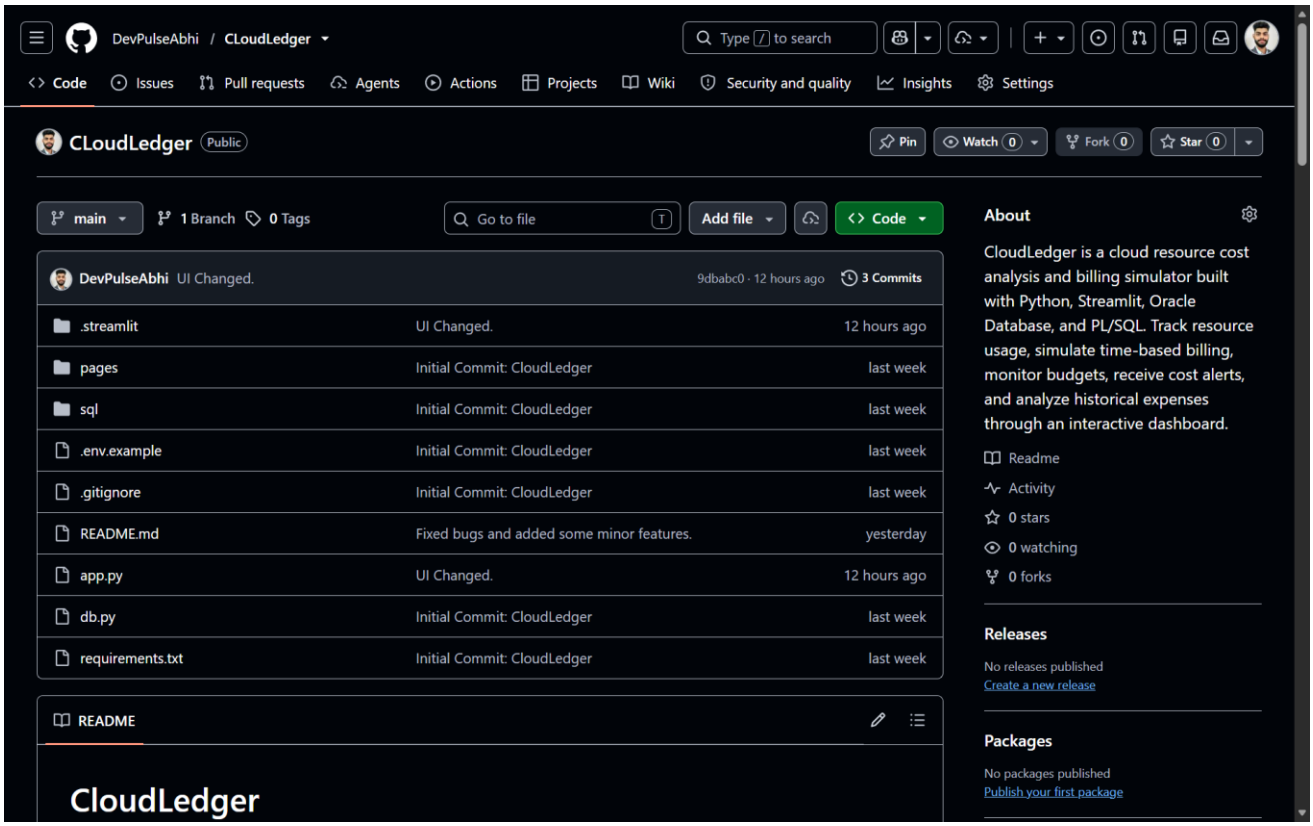


Figure 6. GitHub Repository of CloudLedger

SCAN FOR GITHUB REPO